$myArr; otherwise, we'd be using an invalid index value. The third increment increases $i by one after the iteration has completed (and all statements inside the block have executed).

Notice that $i retains its value after the loop has completed, for the same reasons as the previous example. All variables initialized in a for expression follow PHP's scoping rules and will have the same scope as $myArr and $count. It's quite common to reuse a counter like $i within the same block of code, so be aware that it persists after the loop is complete *and* now has an invalid value. If we tried to use $myArr[$i] after the loop was complete, we'd see an error like this: PHP Warning: Undefined array key 4. For more detail on how variable scope works, see Chapter 3, "Functions."

Loop Expressions

It is legal syntax for any (or even all) of the expressions of a for statement to be empty or contain multiple expressions separated by a comma. For example:

```
$myArr = ['Alice', 'Rita', 'Sue', 'Bob'];

for ($i = 0, $size = count($myArr); $i < $size; ) {
    print "{$myArr[$i]} is here.\n";
    $i++;
}

Output:
Alice is here.
Rita is here.
Sue is here.
Bob is here.
```

Here we've adapted the previous example, moving the initialization of $size into the first for expression and separating it from the initialization of $i with a comma. Notice that incrementing $i now occurs within the loop's code block rather than in the third for expression.

This isn't particularly useful and adds unnecessary clutter to the loop's code block; it's simply done here as a demonstration. Similarly, we could initialize $i and $size before the for statement, and our expressions could look like this: